

Análise de Desempenho de Servidores RTMP e Transcodificadores no Streaming de Vídeo

Valter Abreu Silva Júnior¹, Prof. Dr. Mário Antônio Meirelles Teixeira²

Departamento de Informática – Universidade Federal do Maranhão (UFMA)

Av. dos Portugueses, 1966 – Cidade Universitária Dom Delgado

65080-805 – São Luís – MA – Brasil

valter.abreu@discente.ufma.br, mario.teixeira@ufma.br

Abstract. This paper presents an experimental study on the performance of RTMP servers and video transcoders in a live streaming platform. The implemented platform combines Nginx-RTMP or SRS for ingestion, FFmpeg or GStreamer for transcoding, and HLS for browser delivery. A $2 \times 2 \times 3$ factorial design was executed with four technology combinations, loads of 1, 10, and 50 synthetic viewers, and three repetitions per treatment. The analysis considers master-playlist startup time, CPU and memory consumption, effective bitrate, HTTP request error rate, and container restarts. The results indicate that SRS made the HLS master playlist available earlier in the evaluated environment, while GStreamer used less memory and, in some combinations, exerted greater CPU pressure. The measured end-to-end latency is treated as an invalidated proxy rather than conclusive evidence, because the benchmark does not yet synchronize publication, segment generation, delivery, and playback timestamps.

Keywords: video streaming; RTMP; transcoding; FFmpeg; GStreamer; performance; HLS.

Resumo. Este artigo apresenta um estudo experimental sobre o desempenho de servidores RTMP e transcodificadores em uma plataforma de streaming de vídeo ao vivo. A plataforma implementada combina Nginx-RTMP ou SRS para ingestão, FFmpeg ou GStreamer para transcodificação e HLS para entrega ao navegador. Foi executado um delineamento fatorial $2 \times 2 \times 3$, com quatro combinações tecnológicas, cargas de 1, 10 e 50 espectadores sintéticos e três repetições por tratamento. A análise considera o tempo de disponibilização da playlist mestre, o consumo de CPU e memória, o bitrate efetivo, a taxa de erros de requisições HTTP e os reinícios do container. Os resultados indicam que o SRS disponibilizou a playlist mestre mais cedo no ambiente avaliado, enquanto o GStreamer consumiu menos memória e, em algumas combinações, exerceu maior pressão sobre a CPU. A latência ponta a ponta medida é tratada como um proxy não validado, pois o benchmark ainda não sincroniza os timestamps de publicação, geração do segmento, entrega e reprodução.

Palavras-chave: streaming de vídeo; RTMP; transcodificação; FFmpeg; GStreamer; desempenho; HLS.

1 Introdução

O streaming de vídeo ao vivo depende de uma cadeia que recebe o conteúdo, processa o fluxo e o disponibiliza aos espectadores com continuidade e atraso controlado. Em uma arquitetura desse tipo, o servidor de ingestão, o transcodificador e a camada de entrega exercem funções diferentes, mas compartilham recursos computacionais e influenciam o tempo necessário para iniciar a reprodução. A literatura de qualidade de experiência mostra que o usuário percebe a combinação de atraso inicial, interrupções, variações de qualidade e estabilidade da sessão, e não uma métrica isolada [Seufert et al. 2015, Barman e Martini 2019, Holland et al. 2022].

Uma estratégia comum é usar RTMP na publicação e HLS na distribuição. O RTMP organiza a transferência de áudio, vídeo e dados em uma sessão de publicação, enquanto o HLS descreve a apresentação por meio de playlists e segmentos acessíveis via HTTP [Adobe Systems Incorporated 2012, Pantos e May 2017]. Essa separação permite que a entrada seja recebida por um servidor especializado e que a saída seja consumida por navegadores e clientes HTTP. Entretanto, a escolha do servidor RTMP e do transcodificador pode alterar o momento de criação das playlists, o consumo de CPU e memória e a estabilidade da entrega.

Este artigo deriva de uma plataforma implementada em Java Spring Boot e React, com execução containerizada por Docker Compose. O projeto permite alternar Nginx-RTMP e SRS na ingestão [Arut 2026, SRS Project 2026], bem como FFmpeg e GStreamer na transcodificação [FFmpeg Project 2026, GStreamer Project 2026]. O benchmark automatiza a criação da transmissão, a publicação de uma fonte sintética, a geração de espectadores HTTP e a coleta de métricas. Assim, as quatro combinações são avaliadas dentro da mesma aplicação, com a mesma matriz de carga e no mesmo host.

O objetivo é comparar o comportamento operacional dessas combinações sob cargas de 1, 10 e 50 espectadores sintéticos. São examinados o tempo de disponibilização da playlist mestre HLS, o consumo de CPU e memória do container RTMP, o bitrate efetivo observado, os erros de requisições HTTP e os reinícios registrados durante a sessão. A principal contribuição é uma comparação experimental reproduzível

e delimitada ao ambiente documentado, acompanhada da identificação explícita das métricas que ainda não permitem inferir a latência percebida pelo espectador.

O recorte é relevante porque a transcodificação para múltiplas representações concentra uma parcela importante do custo da plataforma, enquanto a entrega HLS introduz uma sequência de operações de geração, retenção e leitura de segmentos. A análise proposta não pretende estabelecer um ranking universal das ferramentas. Ela procura responder, em condições controladas, como as implementações configuradas no projeto se comportam e quais trade-offs aparecem entre tempo de inicialização, uso de recursos e estabilidade.

2 Trabalhos Relacionados

Estudos de desempenho em sistemas de streaming normalmente observam atraso, continuidade da reprodução, qualidade percebida, vazão e consumo de recursos. A literatura de streaming adaptativo destaca que o comportamento percebido pelo usuário resulta da interação entre qualidade de vídeo, variações de bitrate, tempo de início e interrupções [Seufert et al. 2015, Barman e Martini 2019]. O presente trabalho se concentra na infraestrutura que antecede a decisão adaptativa do cliente: ingestão RTMP, transcodificação e geração da saída HLS.

2.1 Streaming ao vivo e qualidade de experiência

Seufert et al. apresentam uma revisão de qualidade de experiência em HTTP Adaptive Streaming e discutem como atraso inicial, rebuffering, mudanças de representação e qualidade audiovisual se combinam na percepção do usuário. Barman e Martini ampliam essa discussão ao revisar modelos de QoE e desafios de avaliação. Para sistemas ao vivo, o RFC 9317 também diferencia latência de rede de atraso de mídia, que inclui codificação, buffering e etapas intermediárias de distribuição [Holland et al. 2022]. Esses trabalhos justificam a separação, adotada neste artigo, entre uma medida operacional de disponibilidade da playlist e uma medida de latência ponta a ponta.

A Recomendação ITU-T P.1203 descreve um modelo paramétrico que combina informações de codificação, atraso inicial, buffering e adaptações para estimar a qualidade audiovisual de uma sessão [ITU-T 2017]. O modelo não é aplicado neste estudo, pois não foram coletados todos os sinais necessários para a sua utilização. Ainda assim, sua definição de qualidade orienta a interpretação: responder à playlist mestre é um evento de infraestrutura, não uma prova de que o vídeo já começou a ser reproduzido sem interrupções.

2.2 Protocolos e ferramentas da cadeia avaliada

O RTMP é descrito pela especificação da Adobe como um protocolo para transporte de áudio, vídeo e dados em uma sessão de comunicação [Adobe Systems Incorporated 2012]. O HLS organiza uma apresentação multimídia em playlists e segmentos, permitindo que clientes solicitem os arquivos por HTTP e selecionem entre diferentes representações [Pantos e May 2017]. Essa combinação é adequada ao recorte do projeto: publicar o fluxo por RTMP, produzir múltiplas variantes e entregá-las por HLS.

FFmpeg oferece uma interface integrada para leitura de entradas, filtragem, codificação, seleção de streams e multiplexação [FFmpeg Project 2026]. GStreamer estrutura o processamento como um pipeline de elementos conectados; o elemento `hlssink2` escreve playlists e fragmentos, mas depende de um servidor HTTP externo para servi-los [GStreamer Project 2026,a]. Nginx-RTMP e SRS também possuem mecanismos de ingestão e integração com HLS, embora suas arquiteturas e formas de acionar o transcodificador sejam diferentes [Arut 2026, SRS Project 2026]. Portanto, os resultados deste artigo devem ser entendidos como resultados das versões e parâmetros usados na implementação, e não como propriedades universais das ferramentas.

2.3 Reprodutibilidade e lacuna experimental

A containerização facilita a repetição de serviços com configurações declaradas, mas não elimina a influência do sistema hospedeiro, da alocação de CPU, da memória, do armazenamento e da rede. A documentação do Docker descreve containers como processos isolados no host e também expõe mecanismos de contabilização e limitação de recursos [Docker Inc. 2026]. Estudos de avaliação de containers mostram, de modo semelhante, que o desempenho depende da carga e da forma como os recursos são compartilhados [Shirinbab et al. 2020].

A lacuna abordada é a comparação controlada entre dois servidores RTMP e dois transcodificadores dentro da mesma plataforma, com a mesma fonte sintética, o mesmo protocolo de entrega e três níveis de espectadores. Os trabalhos consultados oferecem fundamentos para QoE, protocolos, ferramentas e

reprodutibilidade, mas não respondem qual combinação implementada neste projeto apresenta melhor comportamento sob a matriz definida. A contribuição experimental está em preencher esse recorte com dados brutos, agregação por tratamento e uma discussão que explicita as limitações da instrumentação.

Dao et al. organizam uma visão abrangente dos sistemas de live video streaming e destacam o papel de transcodificação, distribuição adaptativa e recursos computacionais na operação do serviço [Dao et al. 2022]. Essa perspectiva é especialmente relevante aqui, pois uma única publicação resulta em quatro representações HLS e cada representação é processada por uma cadeia de codificação. A literatura, contudo, não substitui a medição local: pequenas diferenças de codec, bitrate, resolução, versão e hardware podem alterar o perfil observado.

3 Metodologia

Foi adotada uma abordagem quantitativa, experimental e comparativa. O fator servidor possui dois níveis (Nginx-RTMP e SRS), o fator transcodificador possui dois níveis (FFmpeg e GStreamer) e a carga possui três níveis (1, 10 e 50 espectadores sintéticos). O delineamento fatorial $2 \times 2 \times 3$ resulta em 12 tratamentos, cada um repetido três vezes, totalizando 36 execuções. A unidade experimental é uma sessão completa de publicação, transcodificação e leitura HLS executada em um ambiente reconstruído pelo Docker Compose.

Tabela 1: Fatores e níveis do experimento

Fator	Níveis	Papel no experimento
Servidor RTMP	Nginx-RTMP; SRS	Recebe a publicação RTMP e dispara o processamento
Transcodificador	FFmpeg; GStreamer	Gera as variantes HLS
Carga	1; 10; 50 viewers	Simula demanda crescente de leitura
Repetição	3 por tratamento	Permite estimar média e desvio padrão

3.1 Arquitetura e fluxo experimental

No benchmark, uma fonte sintética é gerada pelo container `jrottenberg/ffmpeg:6.0-alpine` com o padrão `testsrc` em 1280×720 a 30 quadros por segundo, acompanhada por um sinal de áudio senoidal. O publicador envia essa fonte por RTMP ao servidor selecionado. Nginx-RTMP ou SRS aciona o script do transcodificador, que lê o fluxo RTMP, codifica quatro variantes e grava playlists e segmentos HLS. Os viewers sintéticos são containers `curlimages/curl:8.7.1`; cada um requisita a playlist mestre, uma playlist de variante e o segmento mais recente em ciclos de um segundo. O backend Spring Boot, PostgreSQL, RabbitMQ e frontend fazem parte da aplicação, mas o alvo principal da medição de CPU e memória é o container RTMP/transcodificador.

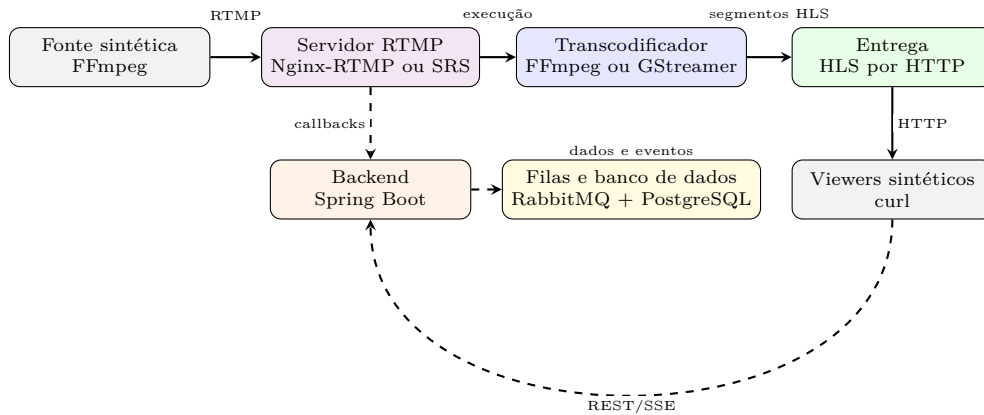


Figura 1: Arquitetura experimental da plataforma.

3.2 Interface da plataforma

Além da infraestrutura de processamento, a plataforma possui uma interface web para criação de transmissões, consulta das lives disponíveis e acesso ao painel do publicador. A interface não é utilizada como instrumento primário de medição, mas demonstra o fluxo de uso que conecta a implementação experimental ao cenário de aplicação. O publicador cria uma transmissão, recebe os dados necessários para configurar o software de captura e acompanha o estado da sessão; o espectador acessa a transmissão pela página de reprodução.

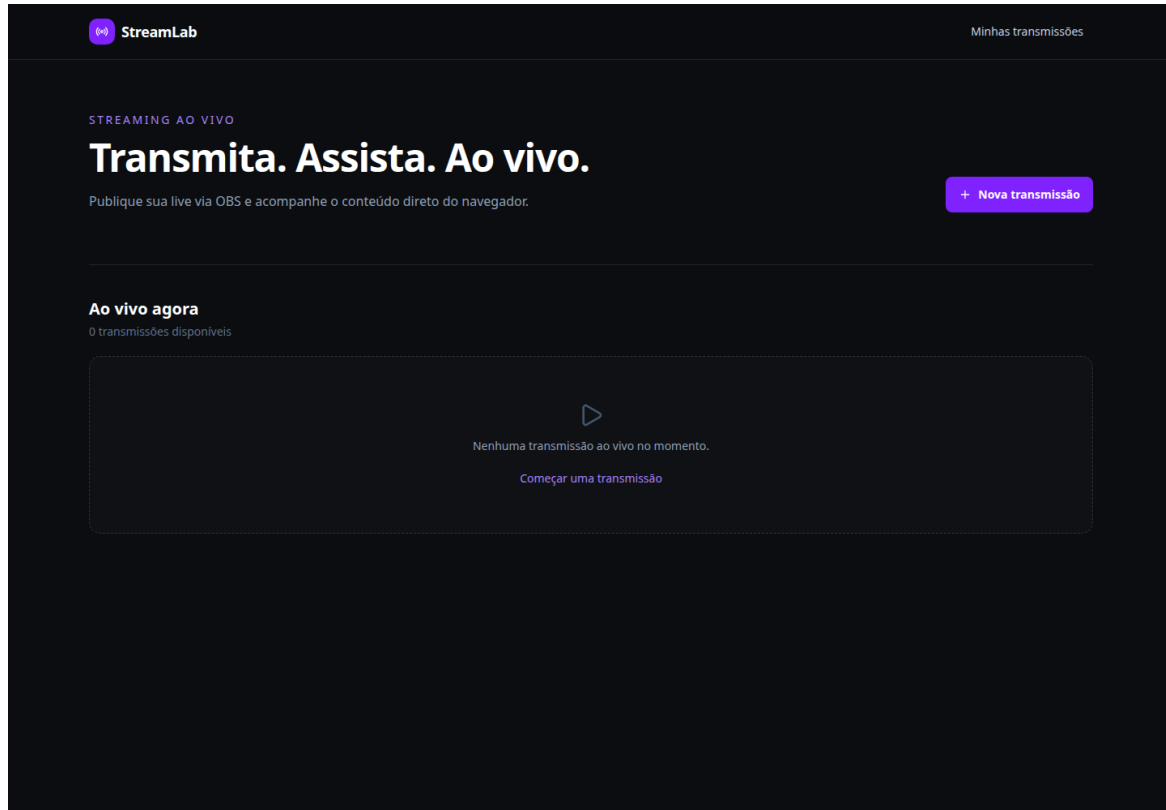


Figura 2: Tela inicial da plataforma de streaming.

A Figura 2 apresenta a tela inicial em seu estado sem transmissões ativas. A interface documenta a aplicação construída, mas não é usada como instrumento primário do benchmark. Dessa forma, os resultados quantitativos são produzidos pelos scripts automatizados, enquanto a figura evidencia a integração entre a infraestrutura avaliada e o cenário de uso da plataforma.

3.3 Configuração do conteúdo e dos pipelines

O FFmpeg gera quatro variantes (1080p, 720p, 480p e 360p) por meio de divisão do vídeo, redimensionamento, codificação H.264 e mux HLS [FFmpeg Project 2026]. Os alvos de vídeo são, respectivamente, 5000, 2800, 1400 e 800 kbps, com áudio de 128, 128, 96 e 96 kbps. O GStreamer executa quatro pipelines paralelos com `rtmpsrc`, decodificação, redimensionamento, `x264enc`, codificação AAC e `hls sink2` [GStreamer Project 2026,a]; seus alvos de vídeo são 5000, 2500, 1400 e 600 kbps, com os mesmos alvos de áudio. Portanto, os alvos agregados não são idênticos: aproximadamente 10,0 Mbps de vídeo no FFmpeg e 9,5 Mbps no GStreamer. Essa diferença é registrada como uma ameaça à comparabilidade e impede interpretar o bitrate como uma medida isolada de eficiência ou qualidade.

Em ambos os pipelines, a duração nominal dos segmentos HLS é de 6 segundos e a playlist mantém 10 segmentos. A codificação é realizada em CPU, sem aceleração por GPU. As quatro representações seguem o modelo de múltiplas qualidades do HLS, mas a ausência de uma normalização completa dos parâmetros significa que a comparação é entre as configurações implementadas no projeto.

3.4 Métricas

Tabela 2: Métricas coletadas e interpretação

Métrica	Unidade	Definição operacional
Startup da playlist mestre	s	Tempo até a playlist mestre responder HTTP 200
CPU média/máxima	%	Uso agregado do container RTMP durante a janela medida
Memória média/máxima	MB	Memória observada no container
Bitrate efetivo	kbps	Estimativa a partir dos segmentos HLS
Taxa de erros HTTP	%	Requisições de playlist ou segmento com erro divididas pelo total
Reinícios	contagem	Diferença do contador de reinícios do container
Primeiro segmento / proxy E2E	s	Campo coletado, mas não validado por timestamps sincronizados

3.5 Procedimento e análise

Cada cenário derruba a execução anterior e sobe uma instância limpa do ambiente containerizado, cuja finalidade é isolar e reproduzir os serviços da aplicação [Docker Inc. 2026]. Em seguida, cria-se uma transmissão, inicia-se a fonte sintética, mede-se o tempo até a playlist mestre responder, inicia-se a quantidade definida de viewers e coleta-se o estado do container com `docker stats` em intervalos de dois segundos. A sessão nominal dura 300 segundos. Os dados brutos são consolidados por uma rotina de agregação estatística, que calcula médias e desvios padrão agrupados por servidor, transcodificador e carga.

Os critérios operacionais adotados no benchmark são disponibilidade da playlist mestre em até 24 s, CPU média de até 240% para 1 e 10 viewers e 280% para 50 viewers, taxa de erros de até 1% para 1 e 10 viewers e 3% para 50 viewers, além de zero reinícios. Esses limites são critérios de aceitação do experimento local, não requisitos universais de streaming. O percentual de CPU é agregado pelo Docker e pode ultrapassar 100% quando o processo utiliza múltiplas threads do host.

3.6 Delineamento fatorial

O experimento pode ser representado como um delineamento fatorial $2 \times 2 \times 3$. Os dois primeiros fatores são tecnológicos e o terceiro representa a carga. Essa organização permite observar tanto o efeito isolado de cada fator quanto a interação entre servidor e transcodificador. Por exemplo, um transcodificador pode apresentar determinado consumo quando associado ao Nginx e comportamento diferente quando associado ao SRS, ainda que a carga permaneça constante.

Para cada tratamento, as três repetições são executadas com a mesma configuração de entrada e duração nominal. A média representa o comportamento central, enquanto o desvio padrão indica a variação entre repetições. Como o número de repetições é pequeno, os desvios são usados de forma descritiva; não se pretende realizar inferência estatística forte ou atribuir significância causal às diferenças observadas.

3.7 Controle de variáveis

Foram mantidos constantes o host, o sistema operacional, o mecanismo de virtualização, a resolução da fonte sintética, a duração dos segmentos e os parâmetros de retenção da playlist. A variável independente principal é a combinação servidor–transcodificador; o número de espectadores funciona como fator de pressão. Como variáveis dependentes, foram observados o startup HLS, o uso de CPU e memória, o bitrate efetivo, a taxa de erros e os reinícios.

Essa separação é importante porque o bitrate produzido não deve ser interpretado isoladamente como indicador de eficiência. Ele também depende do número de variantes, das resoluções, dos bitrates-alvo e dos parâmetros do encoder. Assim, a comparação de consumo precisa ser lida junto com a configuração de saída e com a estabilidade da entrega.

3.8 Ambiente experimental

Tabela 3: Características do ambiente experimental

Componente	Configuração
Sistema operacional	Ubuntu 24.04.3 LTS, arquitetura x86_64
Processador	Intel Core i5-13420H, 13ª geração, 12 threads
Memória	24 GB de RAM
Armazenamento	SSD NVMe, aproximadamente 231 GB
Rede	Loopback e bridge do ambiente containerizado
Orquestração	Docker Engine e Docker Compose
Codificação	CPU, sem aceleração por GPU

O registro do ambiente é necessário porque os valores de CPU e memória não são propriedades isoladas dos servidores ou transcodificadores. O percentual de CPU observado no container pode ultrapassar 100% quando mais de uma thread é utilizada, enquanto a memória deve ser interpretada em relação à capacidade total do host e aos demais serviços ativos. Essa preocupação é consistente com estudos de avaliação de containers, que tratam a infraestrutura e o tipo de carga como fatores inseparáveis da medição de desempenho [Shirinbab et al. 2020].

3.9 Reprodutibilidade

Uma execução pode ser reproduzida seguindo a sequência de selecionar o perfil do servidor, selecionar o transcodificador, reconstruir os containers, criar a transmissão, iniciar a fonte sintética, aplicar a carga de viewers, coletar as séries temporais e repetir o tratamento. O benchmark mantém CSV bruto, logs por cenário, amostras de métricas e um CSV agregado. A execução analisada foi identificada pelo run id `bench-full-20260517-164317`; o estado do host e os parâmetros do pipeline devem ser mantidos junto desses artefatos para permitir auditoria.

3.10 Validade e limitações

Os resultados são internos ao host Ubuntu 24.04.3, Intel Core i5-13420H, 24 GB de RAM, SSD NVMe e Docker Engine. Não há rede externa, GPU, variação de codec, variação de resolução de entrada ou comparação com outros hosts. A latência ponta a ponta não foi instrumentada de forma válida: o script reutiliza a medição de primeiro segmento como proxy e os registros apresentam zeros na maior parte dos tratamentos. A playlist mestre responder não significa necessariamente que um segmento válido já esteja disponível nem que o player tenha iniciado a reprodução. Por isso, essa métrica não é usada como evidência conclusiva; trabalhos futuros deverão sincronizar timestamps entre publicação, criação do segmento, disponibilização HTTP e leitura pelo viewer.

4 Resultados e Discussão

Esta seção apresenta os resultados agregados das três repetições realizadas para cada tratamento experimental. Os valores reportados são médias; os desvios padrão foram calculados a partir das respectivas repetições. Os registros brutos foram preservados para auditoria e reprodutibilidade do experimento. A palavra *startup*, quando usada nas tabelas e figuras, significa o tempo até a playlist mestre responder HTTP 200; ela não representa a latência ponta a ponta da sessão.

Tabela 4: Médias agregadas dos tratamentos

Servidor	Transcod.	Viewers	Playlist (s)	CPU (%)	RAM (MB)	Bitrate (kbps)	Erros HTTP (%)
Nginx	FFmpeg	1	21,67	177,88	1658,06	3524,66	0,00
Nginx	FFmpeg	10	22,00	159,35	1679,64	3526,08	0,00
Nginx	FFmpeg	50	22,33	196,86	1640,75	3522,54	0,00
Nginx	GStreamer	1	22,00	233,44	657,02	5344,59	0,00
Nginx	GStreamer	10	22,00	243,48	623,47	5342,92	0,00
Nginx	GStreamer	50	22,00	292,13	629,52	5344,05	0,00
SRS	FFmpeg	1	8,00	195,32	1665,30	3155,57	0,00
SRS	FFmpeg	10	8,00	197,34	1678,86	3156,01	0,00
SRS	FFmpeg	50	8,00	215,74	1623,08	3157,79	0,00
SRS	GStreamer	1	1,00	207,80	642,26	5340,31	0,00
SRS	GStreamer	10	1,00	215,82	625,27	5338,16	0,00
SRS	GStreamer	50	1,00	192,35	500,90	5337,81	3,15

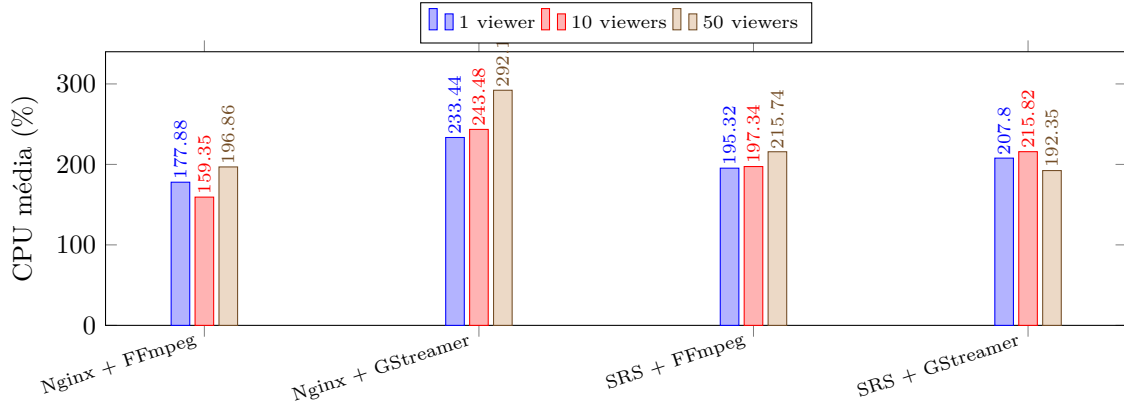


Figura 3: CPU média por combinação tecnológica e carga.

A Figura 3 mostra que a elevação da carga não produz o mesmo comportamento em todas as combinações. Nginx + GStreamer apresenta crescimento contínuo da CPU média, de 233,44% para 292,13%, enquanto SRS + GStreamer apresenta maior variação entre os tratamentos e chega a 192,35% no cenário de 50 viewers. Essa diferença reforça a necessidade de analisar a interação entre servidor e transcodificador, em vez de comparar apenas as médias gerais de FFmpeg e GStreamer.

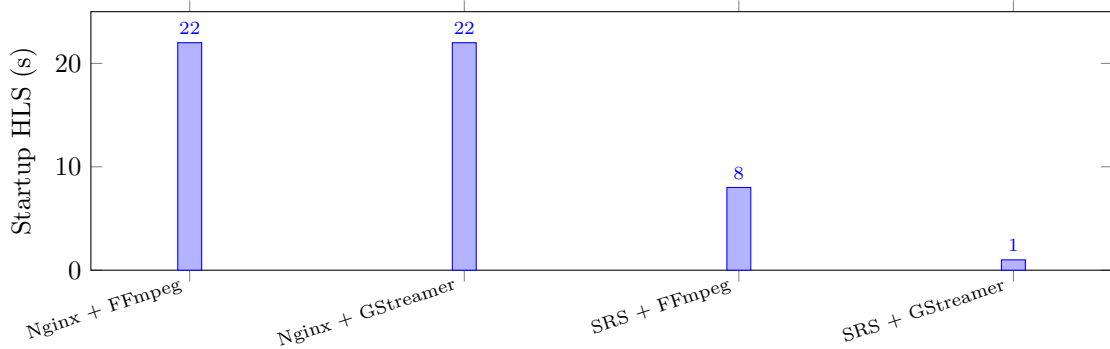


Figura 4: Startup médio da playlist HLS por combinação.

O comportamento da disponibilização da playlist é mais estável que o das métricas de CPU. As três combinações executadas com Nginx permanecem próximas de 22 s, enquanto SRS + FFmpeg fica em 8 s e SRS + GStreamer em 1 s. O padrão sugere um efeito associado ao caminho de inicialização do servidor RTMP, mas a conclusão deve ser limitada ao evento medido. A medição de qualidade percebida exigiria acompanhar também o carregamento de segmentos, o preenchimento do buffer e o início efetivo do vídeo, aspectos tratados em modelos de avaliação de streaming como o ITU-T P.1203 [ITU-T 2017] e nas considerações operacionais do RFC 9317 [Holland et al. 2022].

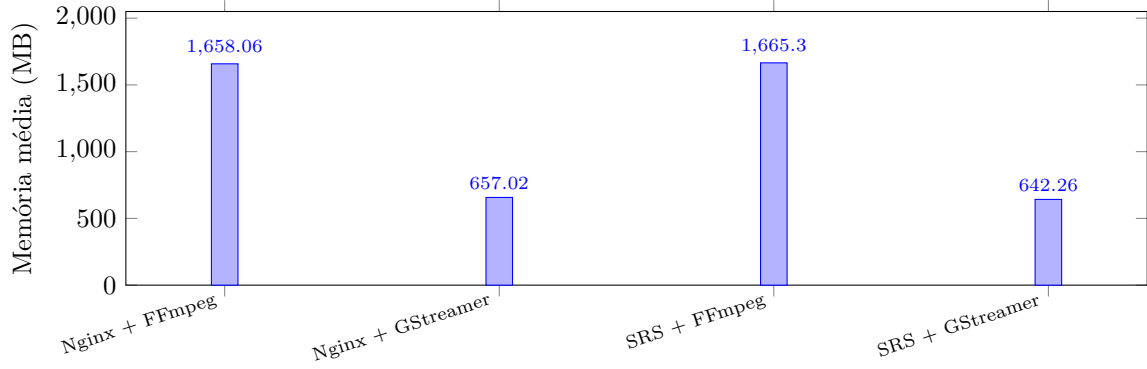


Figura 5: Memória média no cenário de um viewer.

O contraste de memória é consistente no cenário de baixa carga: as combinações com FFmpeg permanecem próximas de 1,66 GB, enquanto as combinações com GStreamer ficam próximas de 0,65 GB. Essa diferença é expressiva, mas não deve ser interpretada como superioridade absoluta do GStreamer. Os pipelines precisam ser comparados com parâmetros equivalentes de qualidade, número de processos e variantes, pois diferenças de implementação podem deslocar o custo entre CPU e memória.

Tabela 5: Desvio padrão de métricas selecionadas nas três repetições

Servidor	Transcodificador	Viewers	Startup (s)	CPU média (%)
Nginx	FFmpeg	1	0,58	3,62
Nginx	FFmpeg	10	0,00	0,56
Nginx	FFmpeg	50	0,58	9,62
Nginx	GStreamer	1	0,00	2,36
Nginx	GStreamer	10	0,00	1,04
Nginx	GStreamer	50	0,00	5,74
SRS	FFmpeg	1	0,00	2,33
SRS	FFmpeg	10	0,00	3,15
SRS	FFmpeg	50	0,00	2,33
SRS	GStreamer	1	0,00	2,55
SRS	GStreamer	10	0,00	4,14
SRS	GStreamer	50	0,00	129,23

A dispersão elevada de CPU no tratamento SRS + GStreamer + 50 viewers merece atenção. Embora a média de 192,35% pareça inferior à de outros cenários, o desvio padrão de 129,23% indica comportamento instável entre repetições. O mesmo tratamento também apresenta desvio padrão de 216,52 MB na memória média e 5,46 pontos percentuais na taxa de erros. Portanto, a média sozinha poderia levar a uma interpretação equivocada; a variabilidade é parte importante da análise de confiabilidade.

4.1 Síntese dos achados

Nos dados atuais, o SRS apresenta tempo de disponibilização da playlist menor que o Nginx nos dois transcodificadores: 8 s contra aproximadamente 22 s com FFmpeg e 1 s contra 22 s com GStreamer. Em termos relativos, isso corresponde a uma redução aproximada de 64% na combinação com FFmpeg e de 95% na combinação com GStreamer. Essas porcentagens descrevem apenas o evento observado pelo benchmark e não uma redução comprovada da latência percebida, pois a instrumentação ponta a ponta é incompleta.

O FFmpeg consome aproximadamente 1,62–1,68 GB de memória média, enquanto o GStreamer fica aproximadamente entre 0,50 e 0,66 GB. Em contrapartida, o GStreamer apresenta maior CPU média com Nginx: 243,48% em 10 viewers e 292,13% em 50 viewers. No maior nível de carga, Nginx + GStreamer utiliza cerca de 48% mais CPU que Nginx + FFmpeg, enquanto SRS + GStreamer tem média menor que SRS + FFmpeg, mas com grande dispersão. Os dados sugerem um trade-off entre memória e CPU, mas não permitem concluir que um transcodificador seja globalmente melhor porque os alvos de bitrate e alguns parâmetros de codificação não são idênticos.

A taxa de erros HTTP foi zero em todos os tratamentos, exceto SRS + GStreamer + 50 viewers, que apresentou 3,15% e 1229 requisições com erro em média. Esse resultado ultrapassa o limite local de 3% e indica degradação sob carga alta. Como o viewer sintético agrega requisições da playlist mestre, da playlist de variante e de segmentos, a métrica não identifica sozinha a causa do erro. É necessário separar os

códigos de resposta e os tipos de recurso nos logs antes de atribuir o problema a segmentos ausentes, janela de playlist, saturação de conexão ou falha transitória do cliente.

4.2 Comparação do startup e do bitrate

O tempo médio de disponibilização da playlist foi de aproximadamente 22 s nos três tratamentos com Nginx, contra 8 s nos tratamentos SRS + FFmpeg e 1 s nos tratamentos SRS + GStreamer. O resultado sugere que o caminho de inicialização do SRS disponibiliza o manifesto mais cedo no ambiente avaliado. Contudo, o startup da playlist não equivale à latência ponta a ponta: o espectador ainda precisa receber segmentos válidos e o player precisa iniciar a reprodução. Essa distinção é coerente com modelos de QoE que tratam o atraso inicial e os eventos de buffering como componentes próprios da qualidade da sessão [ITU-T 2017].

O bitrate efetivo também varia entre as famílias de transcodificador. Os cenários com FFmpeg ficaram próximos de 3,15–3,53 Mbps, ao passo que os cenários com GStreamer ficaram próximos de 5,34 Mbps. Essa diferença não pode ser atribuída apenas ao servidor, pois os pipelines possuem alvos de bitrate distintos e produzem saídas com características diferentes. Um pipeline que gera mais bits por segundo pode oferecer outra relação entre custo computacional e qualidade, mas o presente estudo não realizou avaliação objetiva de qualidade visual.

4.3 Interpretação integrada

A leitura conjunta das métricas aponta três perfis. Nginx + FFmpeg apresenta consumo de memória elevado, CPU intermediária e taxa de erros nula. Nginx + GStreamer reduz a memória, mas passa dos limites de CPU em 10 e 50 viewers. SRS + FFmpeg mantém memória elevada, startup menor e comportamento estável na matriz. SRS + GStreamer combina baixo consumo de memória e startup reduzido, mas apresenta a única taxa de erro acima do critério no maior nível de carga.

Esses perfis mostram que a decisão depende do objetivo operacional. Se a prioridade for reduzir a memória do container, GStreamer é uma alternativa promissora; se a prioridade for manter erros nulos e baixa variabilidade nas métricas observadas, SRS + FFmpeg apresenta o perfil mais equilibrado na matriz analisada. SRS + GStreamer tem o melhor startup da playlist, mas combina esse resultado com erro HTTP acima do limite no maior nível de carga e elevada dispersão. A conclusão é contextual e não substitui testes com outros hosts, codecs, resoluções, bitrates e níveis de audiência. Em sistemas de live streaming, a capacidade computacional e a arquitetura de distribuição são fatores diretamente relacionados à qualidade do serviço [Dao et al. 2022].

4.4 Ameaças à validade

Há ameaças à validade interna, externa e de construção. A validade interna é afetada pela possibilidade de processos do sistema hospedeiro interferirem nas medições de CPU e memória; a mitigação adotada foi executar todos os tratamentos no mesmo host, reconstruir os containers e repetir cada combinação três vezes. A validade externa é limitada porque o estudo utiliza uma única máquina, uma rede local e uma única fonte sintética. A validade de construção é afetada pela latência ponta a ponta não instrumentada e pela taxa de erros medida a partir de requisições sintéticas, que não representa integralmente a experiência de um usuário humano.

Outra ameaça decorre da assimetria potencial entre os pipelines de FFmpeg e GStreamer. Mesmo quando ambos geram as mesmas resoluções, seus parâmetros de bitrate, número de processos, sincronização e comportamento de buffer podem ser diferentes. A comparação, portanto, deve ser apresentada como avaliação das implementações configuradas no projeto, e não como uma propriedade universal das ferramentas. A repetição em diferentes hosts e a normalização rigorosa dos parâmetros são etapas necessárias para ampliar a validade externa.

4.5 Implicações e controles futuros

A validade dos resultados pode ser ampliada por meio da equalização dos alvos de bitrate e demais parâmetros dos pipelines, do registro das versões exatas das imagens, ferramentas e plugins, da medição da latência ponta a ponta com um timestamp embutido no conteúdo, da coleta de percentis, especialmente p50, p95 e p99, para startup e requisições, e da separação dos erros de playlist, segmentos e conexão. Sem essas medidas, as conclusões permanecem restritas ao desempenho operacional do container e à disponibilidade da playlist HLS.

5 Conclusão e Trabalhos Futuros

Este artigo apresentou uma avaliação experimental de servidores RTMP e transcodificadores em uma plataforma de streaming de vídeo ao vivo. A implementação permitiu comparar Nginx-RTMP e SRS combinados com FFmpeg e GStreamer sob cargas de 1, 10 e 50 viewers, com execução automatizada, coleta em CSV e agregação estatística de três repetições por tratamento.

Os resultados sustentam três conclusões principais dentro do ambiente estudado. Primeiro, o SRS disponibilizou a playlist mestre HLS mais cedo que o Nginx-RTMP: 8 s contra aproximadamente 22 s com FFmpeg e 1 s contra 22 s com GStreamer. Segundo, o GStreamer apresentou menor consumo de memória média, mas o Nginx + GStreamer apresentou maior pressão de CPU nas cargas de 10 e 50 viewers. Terceiro, SRS + GStreamer apresentou 3,15% de erros HTTP no cenário de 50 viewers, acima do critério local de 3%, além de elevada dispersão de CPU e memória. Assim, o menor tempo de criação da playlist não implica, por si só, maior estabilidade da sessão.

Essas conclusões descrevem as configurações implementadas e não devem ser generalizadas para qualquer hardware, codec ou parâmetro de codificação. O estudo foi executado em um único host, com codificação CPU-only, rede local, uma fonte sintética e alvos de bitrate não totalmente equivalentes entre FFmpeg e GStreamer. A latência ponta a ponta também não está validada: os campos disponíveis funcionam apenas como proxy de primeiro segmento. O resultado central deste estudo é, portanto, uma base experimental reproduzível e uma caracterização inicial de trade-offs, não um ranking universal das tecnologias.

Como continuidade, recomenda-se: normalizar os alvos de bitrate, parâmetros de encoder e número de representações; registrar versões e hashes das imagens; substituir o proxy por timestamps embutidos no conteúdo; coletar p50, p95 e p99; classificar os erros por recurso e código HTTP; testar cargas superiores; repetir a matriz em outros hosts; e avaliar aceleração por GPU. Essas etapas permitirão separar com maior segurança os efeitos do servidor RTMP, do transcodificador e da carga de espectadores.

6 Referências

Referências

- ADOBE SYSTEMS INCORPORATED. *Adobe's Real Time Messaging Protocol*. 2012. Disponível em: <https://veovera.org/docs/legacy/rtmp-v1-0-spec.pdf>. Acesso em: 21 set. 2026.
- PANTOS, R.; MAY, W. *HTTP Live Streaming*. RFC 8216. IETF, 2017. Disponível em: <https://www.rfc-editor.org/rfc/rfc8216>. Acesso em: 21 set. 2026.
- SEUFERT, M. et al. A survey on quality of experience of HTTP adaptive streaming. *IEEE Communications Surveys & Tutorials*, v. 17, n. 1, p. 469–492, 2015. DOI: <https://doi.org/10.1109/COMST.2014.2360940>.
- BARMAN, N.; MARTINI, M. G. QoE modeling for HTTP adaptive video streaming – a survey and open challenges. *IEEE Access*, v. 7, p. 30831–30859, 2019. DOI: <https://doi.org/10.1109/ACCESS.2019.2901778>.
- HOLLAND, J. et al. *Operational Considerations for Streaming Media*. RFC 9317. IETF, 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9317>. Acesso em: 21 set. 2026.
- FFMPEG PROJECT. *FFmpeg Documentation*. Disponível em: <https://ffmpeg.org/documentation.html>. Acesso em: 21 set. 2026.
- GSTREAMER PROJECT. *GStreamer Documentation*. Disponível em: <https://gstreamer.freedesktop.org/documentation/>. Acesso em: 21 set. 2026.
- GSTREAMER PROJECT. *hlssink2*. In: *GStreamer Documentation*. Disponível em: <https://gstreamer.freedesktop.org/documentation/hls/hlssink2.html>. Acesso em: 21 set. 2026.
- ARUT. *nginx-rtmp-module*. GitHub. Disponível em: <https://github.com/arut/nginx-rtmp-module>. Acesso em: 21 set. 2026.
- SRS PROJECT. *SRS: Simple Realtime Server Documentation*. Disponível em: <https://ossrs.io/lts/en-us/>. Acesso em: 21 set. 2026.
- DOCKER INC. *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 21 set. 2026.

- INTERNATIONAL TELECOMMUNICATION UNION. *Recommendation ITU-T P.1203: Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport*. Geneva: ITU, 2017. Disponível em: <https://www.itu.int/rec/T-REC-P.1203-201710-I/en>. Acesso em: 21 set. 2026.
- DAO, N.-N. et al. A contemporary survey on live video streaming from a computation-driven perspective. *ACM Computing Surveys*, v. 54, n. 10s, art. 202, 2022. DOI: <https://doi.org/10.1145/3519552>.
- SHIRINBAB, S.; LUNDBERG, L.; CASALICCHIO, E. Performance evaluation of containers and virtual machines when running Cassandra workload concurrently. *Concurrency and Computation: Practice and Experience*, v. 32, n. 17, e5693, 2020. DOI: <https://doi.org/10.1002/cpe.5693>.